

**UNIVERSIDADE FEDERAL DA PARAÍBA
CENTRO DE CIÊNCIAS HUMANAS, LETRAS E ARTES
COORDENAÇÃO DO CURSO DE LETRAS PORTUGUÊS**

**MIGUEL MOCHIZUKI SILVA
ARTHUR GOMES
LEUDO NETO**

WIKIART SEARCH:
Aplicação e Processo de Desenvolvimento

João Pessoa
2026

MIGUEL MOCHIZUKI SILVA
ARTHUR GOMES
LEUDO NETO

WIKIART SEARCH:
Aplicação e Processo de Desenvolvimento

Trabalho apresentado como requisito parcial de avaliação da disciplina de Estruturas de Dados e Algoritmos II, do Curso de Licenciatura em Letras – Língua Portuguesa da Universidade Federal da Paraíba.

Orientador: Prof. Dr. Rinaldo de Fernandes

RESUMO

O presente trabalho descreve a aplicação WikiArt Search, um sistema de busca e navegação sobre os metadados do WikiArt, e o seu processo de desenvolvimento. A aplicação é uma engine em C11 que indexa 80.042 obras em duas estruturas de dados (uma tabela ordenada com busca binária e uma árvore afunilada, ou splay tree), servidas por um servidor HTTP e uma interface web que mostra a estrutura trabalhando. A navegação ocorre em três níveis: estilo, artista e obras. Foram implementadas modificações nos algoritmos clássicos para atender à aplicação, como busca de intervalo por chave parcial, busca paginada e ordem de carga embaralhada. O processo de desenvolvimento envolveu a escolha do acervo, a implementação manual das estruturas em C, a integração com o front-end via API de sockets POSIX e a infraestrutura com Docker. Os resultados do benchmark sobre 80.042 obras mostram que a tabela ordenada vence em tempo de resposta em todos os cenários medidos, enquanto a árvore afunilada se adapta ao uso repetido e é mais rápida na inserção com ordem de chegada aleatória. O código-fonte está disponível em <https://github.com/MiguelMochizuki/wikiart-search>.

Palavras-chave: WikiArt; estruturas de dados; tabela ordenada; árvore afunilada; benchmark.

ABSTRACT

This work describes the WikiArt Search application, a search and navigation system over WikiArt metadata, and its development process. The application is a C11 engine that indexes 80,042 artworks in two data structures (an ordered indexed list with binary search and a splay tree), served by an HTTP server and a web interface that shows the structure working. Navigation occurs on three levels: style, artist, and artworks. Modifications to the classical algorithms were implemented to meet the application, such as range search by partial key, paginated search, and shuffled load order. The development process involved choosing the collection, manually implementing the structures in C, integrating with the front-end via a POSIX sockets API, and infrastructure with Docker. Benchmark results over 80,042 artworks show that the ordered table wins in response time in all measured scenarios, while the splay tree adapts to repeated use and is faster on insertion with random arrival order. The source code is available at <https://github.com/MiguelMochizuki/wikiart-search>.

Keywords: WikiArt; data structures; ordered table; splay tree; benchmark.

Sumário

1	INTRODUÇÃO	4
2	ESTRUTURAS DE DADOS	4
3	MODIFICAÇÕES NOS ALGORITMOS CLÁSSICOS	5
4	RESULTADOS	6
5	INTERFACE WEB	7
6	PROCESSO DE DESENVOLVIMENTO	8
7	CONSIDERAÇÕES FINAIS	9
	REFERÊNCIAS	11

1 INTRODUÇÃO

O WikiArt Search é um sistema de busca e navegação sobre os metadados do WikiArt, disponível em <https://github.com/MiguelMochizuki/wikiart-search>. Trata-se de uma engine em C11 que indexa 80.042 obras em duas estruturas de dados, um servidor HTTP e uma interface web que mostra a estrutura trabalhando. O acervo compreende 80.042 obras, 1.119 artistas e 27 estilos, navegado em três níveis: estilo, artista e obras.

A arquitetura organiza-se em três camadas: o navegador (HTML, CSS e JavaScript) comunica-se com um servidor HTTP em C, que utiliza uma engine de busca também em C. A engine mantém duas estruturas de dados, uma linear e uma hierárquica: uma lista indexada ordenada (TabelaOrd, com busca binária) e uma árvore afunilada (ArvoreAfunilada, uma splay tree). Uma interface comum (Buscador, uma vtable) liga as duas estruturas ao servidor e ao benchmark, de modo que uma nova estrutura entra no sistema com uma linha de código.

2 ESTRUTURAS DE DADOS

Cada estrutura mantém três instâncias, uma por nível da navegação. Ambas são genéricas: recebem na criação a ordem entre itens e, a cada busca, um comparador que recorta um intervalo contíguo dessa ordem. O nível 1 corresponde aos gêneros (27 estilos), ordenados por nome. O nível 2 corresponde aos artistas (2.082 pares gênero-artista), ordenados por (gênero, artista). O nível 3 corresponde às obras (80.042 obras), ordenadas por (gênero, artista, id).

A `TabelaOrd` é um array ordenado. A busca é uma busca binária de limite inferior até o começo do intervalo, seguida da coleta enquanto o comparador der zero. Inserir no meio obriga a deslocar a cauda. A `ArvoreAfunilada` é uma árvore binária de busca sem balanceamento explícito. Todo acesso leva o nó acessado até a raiz por rotações (zig, zig-zig e zig-zag). O que foi usado há pouco fica perto do topo, e o custo amortizado de cada operação é $O(\log n)$.

3 MODIFICAÇÕES NOS ALGORITMOS CLÁSSICOS

Os algoritmos clássicos buscam uma chave exata. A aplicação precisa de blocos inteiros, em páginas, e sem que a árvore vire uma lista. As principais modificações foram:

- Busca de intervalo por chave parcial: o comparador devolve menor que zero, zero ou maior que zero contra uma chave parcial, e um bloco inteiro sai numa busca.
- Tamanho da subárvore em cada nó: as rotações mantêm $\text{tam} = 1 + \text{tam}(\text{esq}) + \text{tam}(\text{dir})$, permitindo calcular o total do intervalo e a posição do primeiro item de uma página em $O(\text{altura})$.
- Busca paginada: devolve só uma fatia do intervalo, com o nó de partida obtido por seleção na árvore.
- Afunilamento ascendente sem recursão: ponteiros para o pai permitem afunilar, percorrer em ordem e liberar sem pilha.
- Ordem de carga embaralhada: evita que a árvore vire uma lista encadeada quando o CSV vem agrupado por artista.

4 RESULTADOS

O benchmark foi realizado sobre 80.042 obras, com 10.000 buscas por operação, em três cenários: consultas aleatórias, consultas com localidade e carga clássica (sem embaralhamento).

Nas consultas aleatórias, a tabela ordenada apresenta tempo médio de 0,09 μ s para busca de gênero, 1,21 μ s para busca de artistas de um gênero, 4,53 μ s para busca de todas as obras de um artista e 0,84 μ s para página de 30 obras. A árvore afunilada apresenta, respectivamente, 0,15 μ s, 2,17 μ s, 11,81 μ s e 2,12 μ s. Na busca das obras de um artista, as duas estruturas fazem quase o mesmo número de comparações (270,00 na tabela e 277,40 na árvore), mas o tempo difere: 4,53 μ s na tabela contra 11,81 μ s na árvore, ou seja, a tabela é 2,6 vezes mais rápida. A diferença se explica pela localidade de memória.

Pedir só 30 obras, em vez de coletar o intervalo inteiro, faz a árvore cair de 11,81 para 2,12 μ s (5,6 vezes) e a tabela de 4,53 para 0,84 μ s (5,4 vezes). Na comparação entre carga embaralhada e carga ordenada, a carga ordenada é muito mais barata nas duas estruturas (32 vezes na árvore, 13 vezes na tabela), mas o embaralhamento evita que a árvore comece como uma lista encadeada.

Nas consultas com localidade, a árvore se adapta: as rotações por busca caem a um terço (10,68 para 3,47) e a busca de obras fica 2,1 vezes mais rápida (11,81 para 5,75 μ s). A tabela também melhora (4,53 para 3,02 μ s) e continua mais rápida. Na inserção com carga embaralhada, a árvore carrega os três níveis em 94 ms contra 231 ms da tabela, 2,5 vezes mais rápida.

Em resumo, a tabela vence em tempo de resposta em todos os cenários medidos, só a árvore se reorganiza com o uso, e na inserção com ordem de chegada aleatória a árvore vence.

5 INTERFACE WEB

A navegação é em três níveis: estilo, artista e obras. O seletor no topo troca a estrutura e refaz a consulta da tela aberta, e toda tela mostra as métricas da busca (estrutura, algoritmo, tempo, comparações e rotações). Na tabela ordenada, cada tela é uma grade, com estilos e artistas mostrados pela pintura de capa. Na árvore afunilada, a tela desenha a vista dos quatro primeiros níveis da árvore (1 + 2 + 4 + 8 nós). Clicar num nó o afunila até a raiz, e a engine devolve a árvore já reorganizada.

A API expõe as rotas GET /api/generos, GET /api/artistas, GET /api/busca, GET /api/comparar e GET /api/status.

O parâmetro ed é tabela_ord (padrão) ou arvore_afunilada. A API só atende navegadores vindos de origens autorizadas, definidas em WIKIART_ORIGENS.

6 PROCESSO DE DESENVOLVIMENTO

O desenvolvimento seguiu cinco etapas. Na base de dados, selecionamos o WikiArt conforme a sugestão feita em sala, um acervo multimídia grande, com 80.042 obras. Nas estruturas de dados, escolhemos a árvore afunilada e a tabela ordenada porque ambas permitem uma busca mais eficiente do que a linear.

Na engine, implementamos manualmente, em C, os cabeçalhos e o código-fonte das estruturas. Os testes foram implementados com o uso de IA generativa. No front-end, a integração com a engine pela API de sockets POSIX foi decidida para manter a integração em baixo nível. Na infraestrutura, o Docker e a configuração do projeto Python foram escritos com o auxílio de IA generativa.

As principais dificuldades foram pensar nas modificações das estruturas de dados e avaliar, de forma realista, o impacto delas no sistema.

7 CONSIDERAÇÕES FINAIS

O WikiArt Search compara duas estruturas de dados clássicas sobre um acervo de 80.042 obras. A tabela ordenada vence em tempo de resposta em todos os cenários medidos, sobretudo por localidade de memória. A árvore afunilada se adapta ao uso repetido e é mais rápida na inserção com ordem de chegada aleatória. A interface web permite comparar, na mesma busca, as comparações, as rotações e o tempo de cada estrutura.

REFERÊNCIAS

- [1] SILVA, Miguel Mochizuki; GOMES, Arthur; NETO, Leudo. **WikiArt Search**: sistema de busca sobre metadados do WikiArt com engine em C, benchmark comparativo de três estruturas de dados e interface web. 2026. Disponível em: <https://github.com/MiguelMochizuki/wikiart-search>. Acesso em: 6 out. 2026.